

Below are **copy/paste-ready Sprint prompts for Cursor**, each with **code scaffolding** (SQL migrations + TypeScript interfaces + API route stubs + UI page stubs + jobs + tests).

I'm making one concrete stack assumption so this is executable:

- **Next.js (App Router)**
- **Postgres** via a small pg wrapper (no Prisma/Supabase client assumptions)
- **Vitest** for tests
- Auth/permissions are **stubbed** behind `requireRole()` so Cursor can wire into your existing auth.

If your repo differs (Prisma, Supabase, Express), Cursor can translate the same structure.

SPRINT 0 — Foundations & Guardrails

Paste this into Cursor as one task:

None

Sprint 0 - Foundations & Guardrails

Implement the following scaffolding in the repo:

0. Create folders

- src/lib/success/
- src/app/(settings)/settings/success-measurement/
- src/app/(settings)/settings/integrations/pendo/
- src/app/epics/[epicId]/(tabs)/success-scorecard/
- src/app/epics/[epicId]/(tabs)/retro/
- src/app/api/settings/success-measurement/
- src/app/api/settings/integrations/
- src/app/api/epics/[epicId]/success/

1. Feature flag

```
Create `src/lib/flags.ts` with `isEnabled(flag: string):
boolean`.
```

```
## 2. Permission constants + auth guard (stub)
Create:
```

```
### src/lib/auth/roles.ts
```ts
export type Role = 'ADMIN' | 'PM' | 'PMM' | 'CS' | 'EXEC';
```

### **src/lib/auth/requireRole.ts**

None

```
import type { Role } from './roles';

export function requireRole(userRoles: Role[], allowed: Role[]) {
 const ok = allowed.some(r => userRoles.includes(r));
 if (!ok) {
 const err = new Error('Forbidden');
 // @ts-ignore
 err.status = 403;
 throw err;
 }
}
```

### **src/lib/auth/getUser.ts (stubbed)**

None

```
import type { Role } from './roles';

export async function getUser() {
 // TODO: wire into your auth/session.
 return { id: '00000000-0000-0000-0000-000000000000', roles:
['ADMIN'] as Role[] };
}
```

### 3. Audit log helper (used later)

Create src/lib/audit.ts:

```
None
export async function auditLog(event: {
 actorUserId: string;
 action: string;
 entityType: string;
 entityId: string;
 before?: unknown;
 after?: unknown;
 meta?: Record<string, unknown>;
}) {
 // TODO: wire to DB table or logging. For now console.
 console.info('[AUDIT]', JSON.stringify(event));
}
```

### 4. Add a guard that hides pages when flag disabled

Create src/lib/featureGate.tsx:

```
None
import { isEnabled } from './flags';

export function requireFlag(flag: string) {
 if (!isEnabled(flag)) {
 const err = new Error('Not Found');
 // @ts-ignore
 err.status = 404;
 throw err;
 }
}
```

## Acceptance checks

- Build passes
- Nothing breaks with feature flag false

None

---

## SPRINT 1 – Core Data Model & Migrations (+ Seed)

**\*\*Paste into Cursor:\*\***

```md

Sprint 1 – DB schema + seeds

1) Add DB connection wrapper

Create `src/lib/db.ts` using `pg`:

```ts

import { Pool } from 'pg';

```
const pool = new Pool({
 connectionString: process.env.DATABASE_URL,
 max: 10,
});
```

```
export async function query<T = any>(text: string, params: any[]
= []): Promise<{ rows: T[] }> {
 return pool.query(text, params);
}
```

## 2) Create migration SQL file

Add db/migrations/20260104\_01\_success\_measurement.sql:

None

-- Adoption benchmarks

```
CREATE TABLE IF NOT EXISTS adoption_benchmarks (
 id uuid PRIMARY KEY,
 name text NOT NULL,
 launch_tier text NOT NULL CHECK (launch_tier IN
('TIER_1', 'TIER_2', 'TIER_3')),
 feature_type text NOT NULL,
 target_persona text NOT NULL,
 horizon_days int[] NOT NULL,
 expected_activation numeric[] NOT NULL,
 expected_usage_depth numeric[] NULL,
 expected_ttfv_days int NULL,
 segment_modifiers jsonb NULL,
 is_default boolean NOT NULL DEFAULT false,
 version int NOT NULL DEFAULT 1,
 created_at timestamptz NOT NULL DEFAULT now(),
 updated_at timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE INDEX IF NOT EXISTS idx_adoption_benchmarks_tier_type
 ON adoption_benchmarks(launch_tier, feature_type);
```

-- Success metrics catalog

```
CREATE TABLE IF NOT EXISTS success_metrics (
 id uuid PRIMARY KEY,
 name text NOT NULL,
 category text NOT NULL CHECK (category IN
('ADOPTION', 'REVENUE', 'RETENTION', 'ENABLEMENT', 'FRICTION')),
 description text NULL,
 measurement_type text NOT NULL CHECK (measurement_type IN
('PERCENTAGE', 'COUNT', 'DURATION', 'BOOLEAN')),
 source text NOT NULL CHECK (source IN
('PENDO', 'SNOWFLAKE', 'MANUAL')),
 pendo_event_id text NULL,
 leading_or_lagging text NOT NULL CHECK (leading_or_lagging IN
('LEADING', 'LAGGING')),
```

```

 thresholds jsonb NOT NULL, -- { TIER_1: {...}, TIER_2: {...},
TIER_3: {...} }
 created_at timestamptz NOT NULL DEFAULT now(),
 updated_at timestamptz NOT NULL DEFAULT now()
);

-- Epic success configs
CREATE TABLE IF NOT EXISTS epic_success_configs (
 epic_id uuid PRIMARY KEY,
 benchmark_id uuid NOT NULL REFERENCES adoption_benchmarks(id),
 post_launch_owner uuid NOT NULL,
 locked boolean NOT NULL DEFAULT false,
 locked_at timestamptz NULL,
 created_at timestamptz NOT NULL DEFAULT now(),
 updated_at timestamptz NOT NULL DEFAULT now()
);

-- Epic metric mapping (3-7 metrics per epic)
CREATE TABLE IF NOT EXISTS epic_success_metrics (
 id uuid PRIMARY KEY,
 epic_id uuid NOT NULL,
 metric_id uuid NOT NULL REFERENCES success_metrics(id),
 threshold_override jsonb NULL,
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(epic_id, metric_id)
);

CREATE INDEX IF NOT EXISTS idx_epic_success_metrics_epic ON
epic_success_metrics(epic_id);

-- Pendo integration (single row for now)
CREATE TABLE IF NOT EXISTS pendo_integrations (
 id uuid PRIMARY KEY,
 api_key_encrypted text NOT NULL,
 environment text NOT NULL DEFAULT 'prod',
 last_sync timestamptz NULL,

```

```

 status text NOT NULL DEFAULT 'disconnected',
 created_at timestamptz NOT NULL DEFAULT now(),
 updated_at timestamptz NOT NULL DEFAULT now()
);

-- Scorecard snapshots
CREATE TABLE IF NOT EXISTS epic_scorecards (
 id uuid PRIMARY KEY,
 epic_id uuid NOT NULL,
 snapshot_date date NOT NULL,
 metric_results jsonb NOT NULL,
 benchmark_comparison jsonb NOT NULL,
 overall_status text NOT NULL CHECK (overall_status IN
('ON_TRACK', 'AT_RISK', 'MISSED')),
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(epic_id, snapshot_date)
);

-- Retrospectives
CREATE TABLE IF NOT EXISTS epic_retros (
 id uuid PRIMARY KEY,
 epic_id uuid NOT NULL,
 day_marker int NOT NULL CHECK (day_marker IN (30,60,90)),
 status text NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING', 'SUBMITTED')),
 outcome text NULL CHECK (outcome IN ('YES', 'PARTIAL', 'NO')),
 blockers text[] NULL,
 assumptions_wrong text NULL,
 repeat_next_time text NULL,
 change_next_time text NULL,
 action_items jsonb NULL,
 submitted_by uuid NULL,
 submitted_at timestamptz NULL,
 created_at timestamptz NOT NULL DEFAULT now(),
 updated_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(epic_id, day_marker)
);

```

```
);
```

### 3) Add TypeScript domain types

Create `src/lib/success/types.ts`:

None

```
export type LaunchTier = 'TIER_1' | 'TIER_2' | 'TIER_3';
export type FeatureType = 'WORKFLOW' | 'UX' | 'ADMIN' | 'AI' |
 'INTEGRATION' | 'REPORTING';
export type Persona = 'RECRUITER' | 'MANAGER' | 'ADMIN' |
 'EMPLOYEE' | 'EXEC';

export type MetricCategory = 'ADOPTION' | 'REVENUE' | 'RETENTION'
 | 'ENABLEMENT' | 'FRICTION';
export type MeasurementType = 'PERCENTAGE' | 'COUNT' | 'DURATION'
 | 'BOOLEAN';
export type MetricSource = 'PENDO' | 'SNOWFLAKE' | 'MANUAL';
export type LeadingLagging = 'LEADING' | 'LAGGING';

export type ScoreStatus = 'ON_TRACK' | 'AT_RISK' | 'MISSED';

export interface AdoptionBenchmark {
 id: string;
 name: string;
 launch_tier: LaunchTier;
 feature_type: string;
 target_persona: string;
 horizon_days: number[];
 expected_activation: number[];
 expected_usage_depth?: number[] | null;
 expected_ttfv_days?: number | null;
 segment_modifiers?: any;
 is_default: boolean;
 version: number;
```

```

}

export interface SuccessMetric {
 id: string;
 name: string;
 category: MetricCategory;
 description?: string | null;
 measurement_type: MeasurementType;
 source: MetricSource;
 pendo_event_id?: string | null;
 leading_or_lagging: LeadingLagging;
 thresholds: any;
}

```

## 4) Add seed script

Create scripts/seed\_success\_measurement.ts (node ts-node) that inserts:

- Default benchmark per tier x feature\_type (start with feature\_type='WORKFLOW')
- 6 default metrics: Activation Rate, Usage Depth, TTFV, CS Tickets (manual), Enablement Completion (manual), Deal Influence (manual)

Use UUIDs with crypto.randomUUID().

## Acceptance

- Migration applies
- Seed runs idempotently (skip if exists)

None

---

## SPRINT 2 – Settings > Success Measurement > Adoption  
Benchmarks (CRUD)

**\*\*Paste into Cursor:\*\***

```md

Sprint 2 – Adoption Benchmark Library (Settings)

1) API routes (App Router)

Create:

###

src/app/api/settings/success-measurement/adoption-benchmarks/route.ts

Implement GET and POST.

Rules:

- GET: allowed roles ADMIN, PM, PMM, CS, EXEC
- POST: ADMIN only
- Validate horizon_days = [30,60,90], expected_activation length matches horizons

```ts

import { NextResponse } from 'next/server';

import { query } from '@lib/db';

import { getUser } from '@lib/auth/getUser';

import { requireRole } from '@lib/auth/requireRole';

export async function GET() {

const user = await getUser();

requireRole(user.roles, ['ADMIN', 'PM', 'PMM', 'CS', 'EXEC']);

const { rows } = await query('SELECT \* FROM adoption\_benchmarks  
ORDER BY launch\_tier, feature\_type, name');

return NextResponse.json(rows);

}

```

export async function POST(req: Request) {
 const user = await getUser();
 requireRole(user.roles, ['ADMIN']);

 const body = await req.json();
 const { id, name, launch_tier, feature_type, target_persona,
horizon_days, expected_activation, expected_usage_depth,
expected_ttfv_days, segment_modifiers, is_default } = body;

 if (!Array.isArray(horizon_days) || horizon_days.join(',') !==
'30,60,90') {
 return NextResponse.json({ error: 'horizon_days must be
[30,60,90]' }, { status: 400 });
 }
 if (!Array.isArray(expected_activation) ||
expected_activation.length !== horizon_days.length) {
 return NextResponse.json({ error: 'expected_activation length
must match horizon_days' }, { status: 400 });
 }

 await query(
 `INSERT INTO adoption_benchmarks
 (id, name, launch_tier, feature_type, target_persona,
horizon_days, expected_activation, expected_usage_depth,
expected_ttfv_days, segment_modifiers, is_default)
 VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)`,
 [id, name, launch_tier, feature_type, target_persona,
horizon_days, expected_activation, expected_usage_depth ?? null,
expected_ttfv_days ?? null, segment_modifiers ?? null,
!!is_default]
);

 return NextResponse.json({ ok: true });
}

```

## src/app/api/settings/success-measurement/adoption-benchmarks/[id]/route.ts

Implement PUT.

Rules:

- ADMIN only
- If benchmark referenced by any locked epic\_success\_configs -> forbid update
- Enforce one default per (launch\_tier, feature\_type). If setting is\_default true, unset others in same pair.

None

```
import { NextResponse } from 'next/server';
import { query } from '@lib/db';
import { getUser } from '@lib/auth/getUser';
import { requireRole } from '@lib/auth/requireRole';

export async function PUT(req: Request, { params }: { params: {
 id: string } }) {
 const user = await getUser();
 requireRole(user.roles, ['ADMIN']);

 const id = params.id;
 const body = await req.json();

 const lockCheck = await query(
 `SELECT 1 FROM epic_success_configs WHERE benchmark_id=$1 AND
locked=true LIMIT 1`,
 [id]
);
 if (lockCheck.rows.length) {
 return NextResponse.json({ error: 'Benchmark is in use by a
locked epic' }, { status: 409 });
 }
}
```

```

 const current = await query(`SELECT launch_tier, feature_type
FROM adoption_benchmarks WHERE id=$1`, [id]);
 if (!current.rows.length) return NextResponse.json({ error:
'Not found' }, { status: 404 });

 const { launch_tier, feature_type } = current.rows[0] as any;

 if (body.is_default === true) {
 await query(
 `UPDATE adoption_benchmarks SET is_default=false,
updated_at=now()
 WHERE launch_tier=$1 AND feature_type=$2 AND id<>$3`,
 [launch_tier, feature_type, id]
);
 }

 await query(
 `UPDATE adoption_benchmarks SET
 name=COALESCE($2,name),
 target_persona=COALESCE($3,target_persona),
 horizon_days=COALESCE($4,horizon_days),
 expected_activation=COALESCE($5,expected_activation),
 expected_usage_depth=COALESCE($6,expected_usage_depth),
 expected_ttfv_days=COALESCE($7,expected_ttfv_days),
 segment_modifiers=COALESCE($8,segment_modifiers),
 is_default=COALESCE($9,is_default),
 version=version+1,
 updated_at=now()
 WHERE id=$1`,
 [id, body.name ?? null, body.target_persona ?? null,
body.horizon_days ?? null, body.expected_activation ?? null,
body.expected_usage_depth ?? null, body.expected_ttfv_days ??
null, body.segment_modifiers ?? null, body.is_default ?? null]
);

 return NextResponse.json({ ok: true });

```

```
}
```

## 2) UI page

Create `src/app/(settings)/settings/success-measurement/adoption-benchmarks/page.tsx`

- Fetch GET API
- Render table with filters (tier, feature\_type)
- Show “Used by X epics”: call new API or extend GET with usage counts (preferred: extend GET query using LEFT JOIN aggregation)
- For Admin: show Edit modal + Default toggle calling PUT.

Keep UI simple using existing components.

## Acceptance

- Admin can edit + set default
- PM can view only
- Locked epic prevents benchmark updates

None

---

## SPRINT 3 – Settings > Success Metrics (CRUD) + Pendo mapping fields

\*\*Paste into Cursor:\*\*

```md

Sprint 3 – Success Metrics (Settings)

1) API routes

Create:

###

src/app/api/settings/success-measurement/success-metrics/route.ts
GET/POST (ADMIN for POST, viewer roles for GET)

###

src/app/api/settings/success-measurement/success-metrics/[id]/route.ts
PUT (ADMIN only)

Validation:

- thresholds must include keys for TIER_1/TIER_2/TIER_3
- if source=PENDO then pendo_event_id required

2) UI

Create

`src/app/(settings)/settings/success-measurement/success-metrics/page.tsx`

- Table list
- Admin edit/create
- Pendo event picker should be a dropdown populated from `/api/settings/integrations/pendo/events` when connected; otherwise show message "Connect Pendo first"

3) Block changes after Epic lock

Not in settings; enforce later at epic mapping.

Acceptance

- Admin can CRUD metrics
 - PM can view only
 - Pendo event dropdown works when integration connected
-

SPRINT 4 — Settings > Integrations > Pendo (MCP read-only)

Paste into Cursor:

None

```
# Sprint 4 - Pendo Integration (Read-only ingestion)

## 1) Create encryption helpers (simple)
Create `src/lib/crypto.ts` with placeholder (Cursor: wire to your KMS later)

```ts
export function encrypt(text: string) { return
Buffer.from(text).toString('base64'); }
export function decrypt(text: string) { return Buffer.from(text,
'base64').toString('utf8'); }
```

## 2) Pendo client

Create src/lib/success/pendoClient.ts (read-only)

- Use fetch to Pendo API endpoints (Cursor: use official Pendo docs already used in repo)
- Provide functions:
  - testConnection(apiKey: string): Promise<boolean>
  - listEvents(apiKey: string): Promise<{ id: string; name: string }[]>

If you cannot hit real Pendo in dev, stub behind env PENDO MOCK=true.

## 3) API routes

Create:

- src/app/api/settings/integrations/pendo/connect/route.ts (POST)

- store encrypted api key in pendo\_integrations (single row)
  - set status=connected
- src/app/api/settings/integrations/pendo/events/route.ts (GET)
  - if disconnected return []
  - else call pendoClient.listEvents
- src/app/api/settings/integrations/pendo/sync/route.ts (POST)
  - update last\_sync

## 4) UI page

Create src/app/(settings)/settings/integrations/pendo/page.tsx

- Form: API key, environment
- Button: Test + Save
- Section: Event preview table
- Health: last\_sync + stale warning if >24h

## Acceptance

- Connection persists in DB
- Event list viewable
- Works with mock mode

None

---

## SPRINT 5 – Epic Success Config + Locking + PM Assignment

**\*\*Paste into Cursor:\*\***

```md

Sprint 5 – Epic Success Config + Locking

1) Epic success config APIs

Create:

- `src/app/api/epics/[epicId]/success/config/route.ts` (POST/GET)
- `src/app/api/epics/[epicId]/success/lock/route.ts` (PUT)

Data rules:

- Must set benchmark_id
- Must set post_launch_owner (default to current user)
- Must map 3-7 metrics in epic_success_metrics
- Locking sets `locked=true, locked\_at=now()`
- After lock: prevent changes to benchmark_id and metric list (unless ADMIN with audit)

Implement helper service `src/lib/success/epicSuccessService.ts` with functions:

- `getConfig(epicId)`
- `upsertConfig(epicId, benchmarkId, postLaunchOwner)`
- `setMetrics(epicId, metricIds)`
- `lock(epicId, actorUserId)`

2) UI: Epic Success Setup (pre-launch)

Add a section in Epic UI (wherever epic editor lives) OR create a page:

`src/app/epics/[epicId]/(tabs)/success-scorecard/setup.tsx`

- Benchmark dropdown (from benchmarks)
- Metrics multi-select (from metrics)
- Post-launch owner picker (users list; if you don't have user lookup, keep as UUID input for now)

3) Auto-lock at Go/No-Go

If you have a Go/No-Go transition endpoint already, integrate:

- when epic status becomes "GO" -> call `lock(epicId)`

If not available, add manual "Lock" button visible to PM/Admin.

Acceptance

- Epic cannot lock unless benchmark + 3-7 metrics + owner set
- After lock, PM cannot modify config
- ADMIN override logs audit

SPRINT 6 — Success Scorecard Tab + Daily Snapshot Job

Paste into Cursor:

None

Sprint 6 – Epic Success Scorecard Tab + Daily snapshots

1) Scorecard compute

Create `src/lib/success/scorecard.ts`

Inputs:

- epicId
- epic_success_configs (benchmark + metrics)
- Pendo event usage (Phase 1: mock or minimal)

Output JSON shape:

```ts

```
export interface ScorecardResult {
 epicId: string;
 asOf: string; // ISO date
 overallStatus: 'ON_TRACK' | 'AT_RISK' | 'MISSED';
 metrics: Array<{
 metricId: string;
```

```

 metricName: string;
 actual: number | boolean | null;
 expected: number | null;
 status: 'ON_TRACK' | 'AT_RISK' | 'MISSED';
 source: 'PENDO' | 'SNOWFLAKE' | 'MANUAL';
 }>;
 adoptionCurve: {
 horizons: number[]; // [30,60,90]
 expectedActivation: number[];
 actualActivation: number[]; // if not available, nulls or
 zeros w/ "dataMissing" flag
 };
 alerts: string[];
}

```

Rules:

- If missing data for a metric -> treat as AT\_RISK and include alert “Missing data for X”
- OverallStatus:
  - MISSED if >=2 metrics MISSED
  - AT\_RISK if any metric AT\_RISK or missing
  - ON\_TRACK otherwise

## 2) API route

Create src/app/api/epics/[epicId]/success/scorecard/route.ts GET:

- viewer roles allowed (ADMIN/PM/PMM/CS/EXEC)
- returns latest snapshot if exists for today, else computes on the fly and persists snapshot for today

Persist to epic\_scorecards with snapshot\_date=today.

### 3) Daily job

Create scripts/cron\_daily\_scorecards.ts:

- Find epics that are locked and launched (you may not have GA date; if not, compute anyway for all locked epics)
- Compute scorecard and upsert snapshot\_date=today

### 4) UI Tab

Create src/app/epics/[epicId]/(tabs)/success-scorecard/page.tsx

- Fetch scorecard endpoint
- Render:
  - Status banner
  - Metrics table (actual vs expected)
  - Simple chart (optional; if no chart lib, render as table)

### Acceptance

- Scorecard loads for an epic
- Snapshot stored daily
- Missing data clearly shown and counts as risk

None

---

## SPRINT 7 – Retro Tab + 30/60/90 Auto-creation

\*\*Paste into Cursor:\*\*

```

```md
# Sprint 7 – Retrospectives (auto-create + submit)

## 1) Retro placeholder creation
We need an Epic GA date. If epic has no GA date, add
`epics.ga_date` (DATE) or use existing release date field.
If not present, create migration:
ALTER TABLE epics ADD COLUMN IF NOT EXISTS ga_date date;

## 2) Job: create placeholders
Create `scripts/cron_retro_placeholders.ts`
- For each epic with ga_date not null:
  - If today == ga_date + 30 -> upsert retro row (day_marker=30,
status=PENDING)
  - same for 60, 90
- Also create retro early if scorecard status MISSED and no 30
retro exists yet (optional)

## 3) Retro APIs
Create:
- `src/app/api/epics/[epicId]/success/retros/route.ts` GET list
- `src/app/api/epics/[epicId]/success/retros/submit/route.ts`
POST

POST body:
```json
{
 "dayMarker": 30,
 "outcome": "YES|PARTIAL|NO",
 "blockers": ["..."],
 "assumptionsWrong": "...",
 "repeatNextTime": "...",
 "changeNextTime": "...",
 "actionItems": [{ "owner": "...", "text": "...", "dueDate":
"YYYY-MM-DD" }]
}

```

```
}
```

Rules:

- Must have scorecard snapshot for today or latest  $\leq 7$  days
- Only PM or Admin can submit
- Once submitted, editing allowed only within 7 days by submitter or Admin

## 4) UI Retro tab

Create `src/app/epics/[epicId]/(tabs)/retro/page.tsx`

- Show list of 30/60/90
- For pending: render structured form (radio outcome, multi blockers, text areas, action items list)
- Submit calls API

## Acceptance

- Retro placeholders appear automatically
- Submission stored and immutable beyond window
- Cannot submit without recent scorecard

None

---

## SPRINT 8 – PM Monitoring Assignment + Reminders + Escalation

\*\*Paste into Cursor:\*\*

```md

Sprint 8 – PM ownership + reminders + escalation

1) Track "reviewed" activity

Add table:

```
CREATE TABLE IF NOT EXISTS epic_success_reviews (  
  id uuid PRIMARY KEY,  
  epic_id uuid NOT NULL,  
  reviewer_user_id uuid NOT NULL,  
  reviewed_at timestamptz NOT NULL DEFAULT now()  
);
```

Add API:

POST /api/epics/[epicId]/success/review (marks reviewed)

2) UI

On Scorecard tab add button "Mark reviewed" (PM/Admin only)

3) Weekly reminder job

Create `scripts/cron\_weekly\_success\_reminders.ts`

- Find epics with locked=true and ga_date within last 90 days
- For each epic, if no review in last 7 days:
 - send notification (for now: console log; if you have email/slack service, wire it)

4) Escalation

If no review for 14 days OR retro overdue by 7 days -> escalate to PM's manager (if manager relationship exists); otherwise log with "ESCALATION_REQUIRED" and list in admin view.

Acceptance

- Reviews recorded
 - Reminder job identifies unreviewed epics
 - Escalation logic runs
-

SPRINT 9 — Permissions + Hardening + Tests

Paste into Cursor:

None

```
# Sprint 9 - Permissions, hardening, and tests
```

```
## 1) Enforce permissions consistently
```

- Settings Benchmarks/Metrics: ADMIN write only
- Epic config: PM/ADMIN write before lock; after lock ADMIN only
- Scorecard: view for PM/PMM/CS/EXEC/ADMIN
- Retro: submit PM/ADMIN, comment PMM/CS (if comment feature exists)

```
## 2) Add Vitest tests
```

```
Create tests in `src/lib/success/__tests__`:
```

- lock_requires_benchmark_metrics_owner
- benchmark_update_blocked_if_locked_epic
- overall_status_calculation
- retro_requires_recent_scorecard
- permissions_forbidden_cases

```
Use a db test harness:
```

- If no test DB exists, mock `query()` with in-memory stubs.

```
## Acceptance
```

- Tests run in CI
- Forbidden operations return 403/409 correctly